Agent Best Practices: From Content Layer to User Interface

by Geert Baeke

Geert Baeke

AI Architect & Tech Lead

Building practical AI solutions with a focus on tech stack and architecture

Company

MBarQ

Cronos

Cronos AI Lab

Blog

blog.baeke.info

LinkedIn

linkedin.com/in/geertbaeke

Agenda

- Content layer : securely accessing corporate data
- Agent layer: the shift to model harnesses
- User Interface layer: streaming content and tool calls

PART 1

Content Layer

Guidelines

Avoid building custom indexes

Use an agent platform that has built-in support for your data sources
E.g. Copilot Studio with out-of-box SharePoint data source

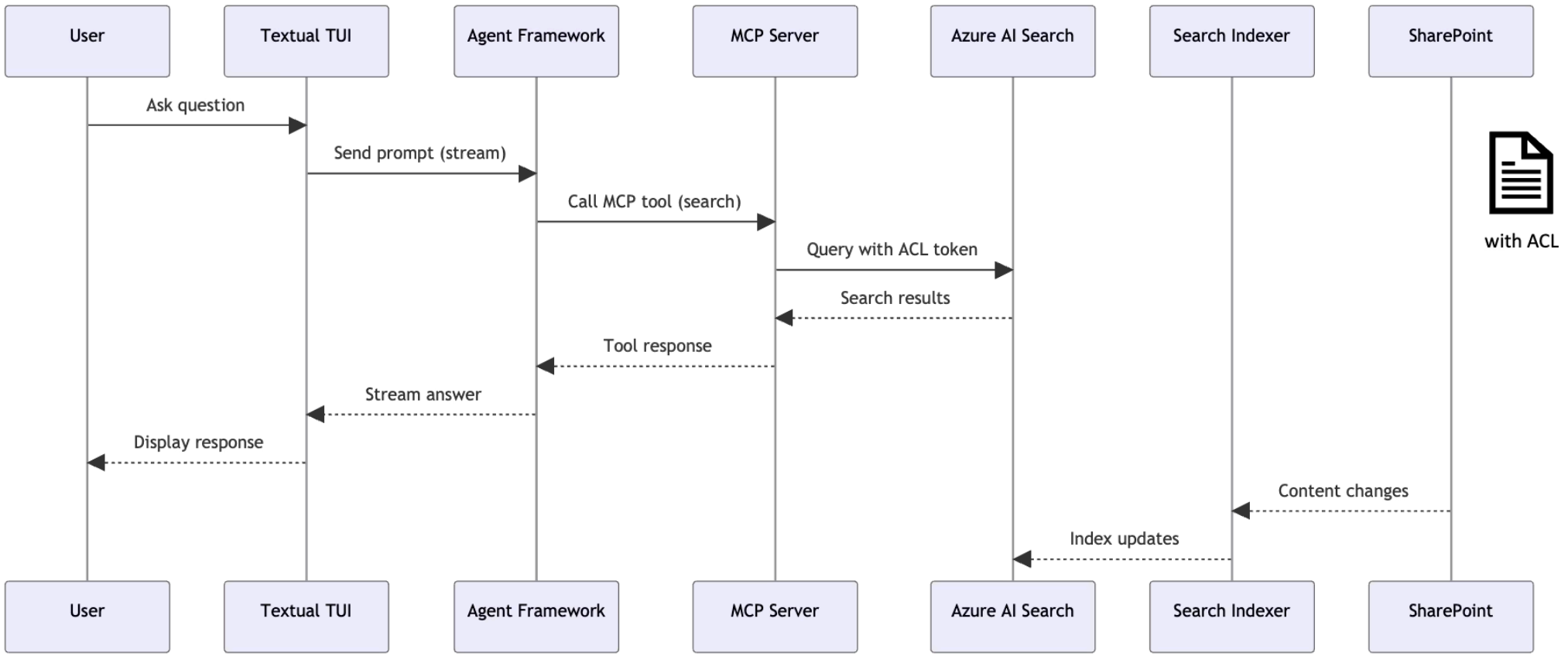
In custom code prefer APIs like Copilot Retrieval APIs

When you build a custom index with Azure AI Search prefer the pull approach

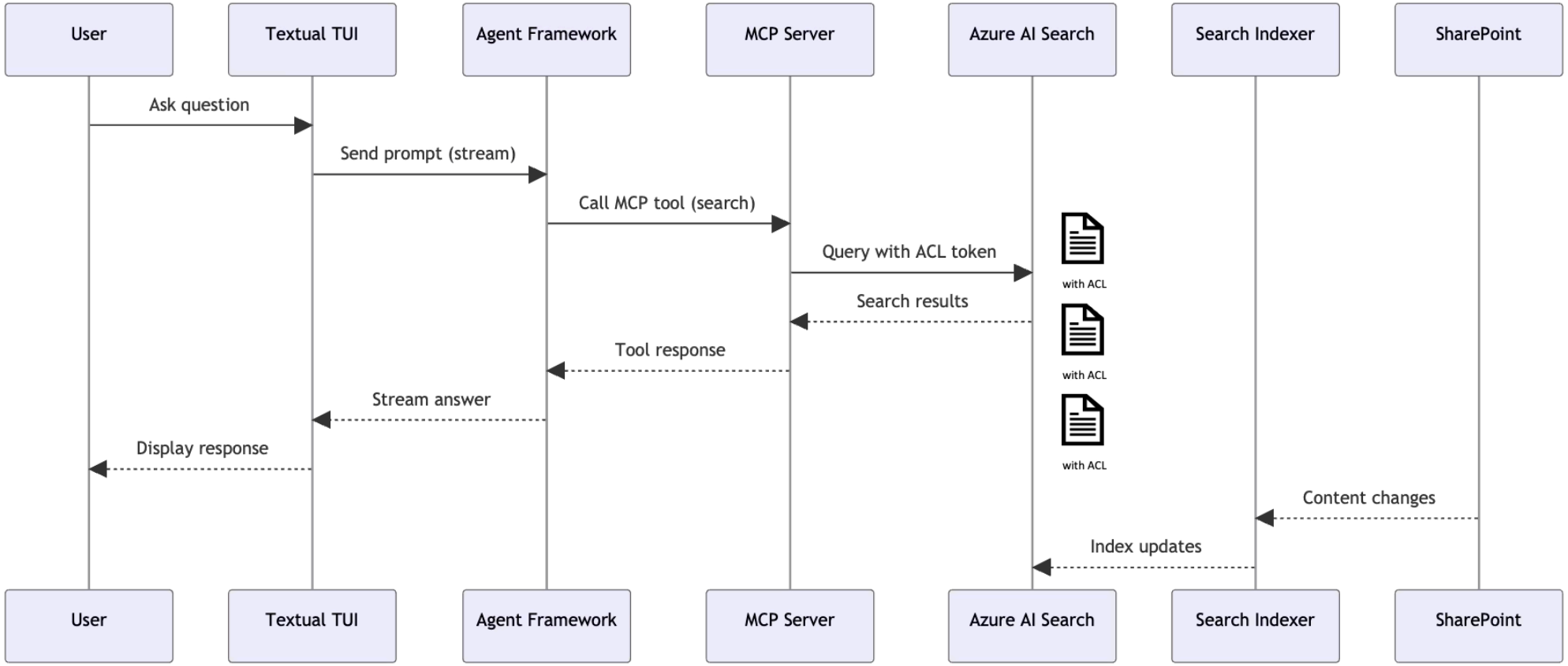
But in most cases...

- Requirements are like:
  - "We have a lot of content so the agent needs to filter based on metadata"
  - "The agent needs to search in custom metadata fields"
  - "The agents wants to find images"
  - "The agent needs to take the user's access control list into account"

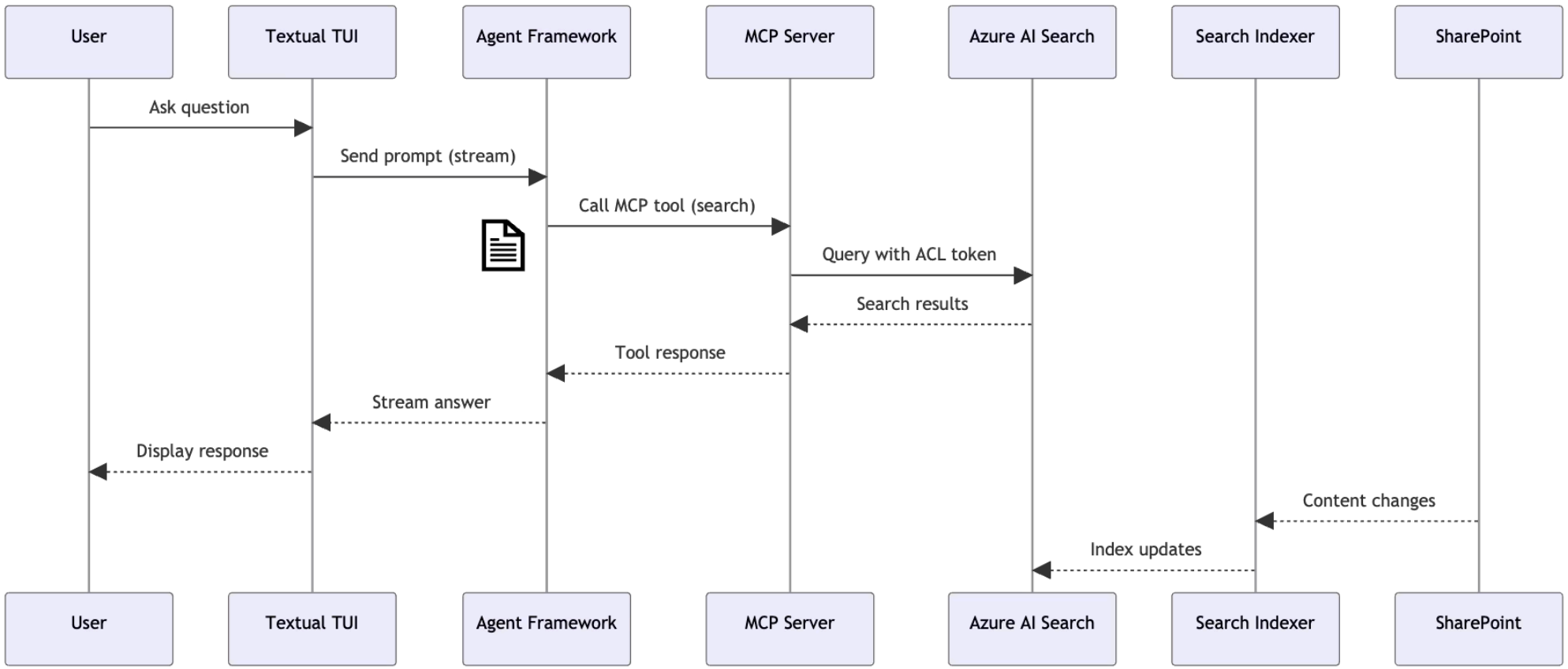
So we build a custom index!



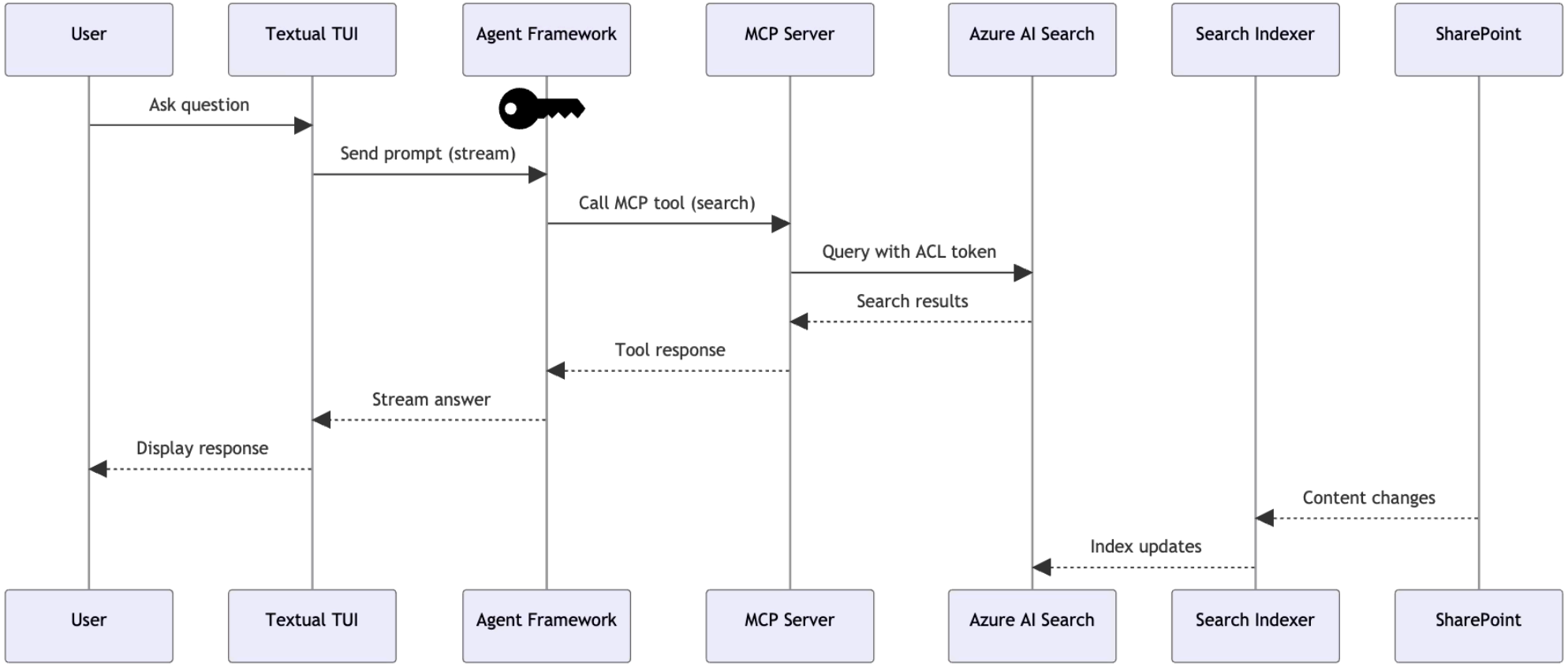
Content in SharePoint with access control lists



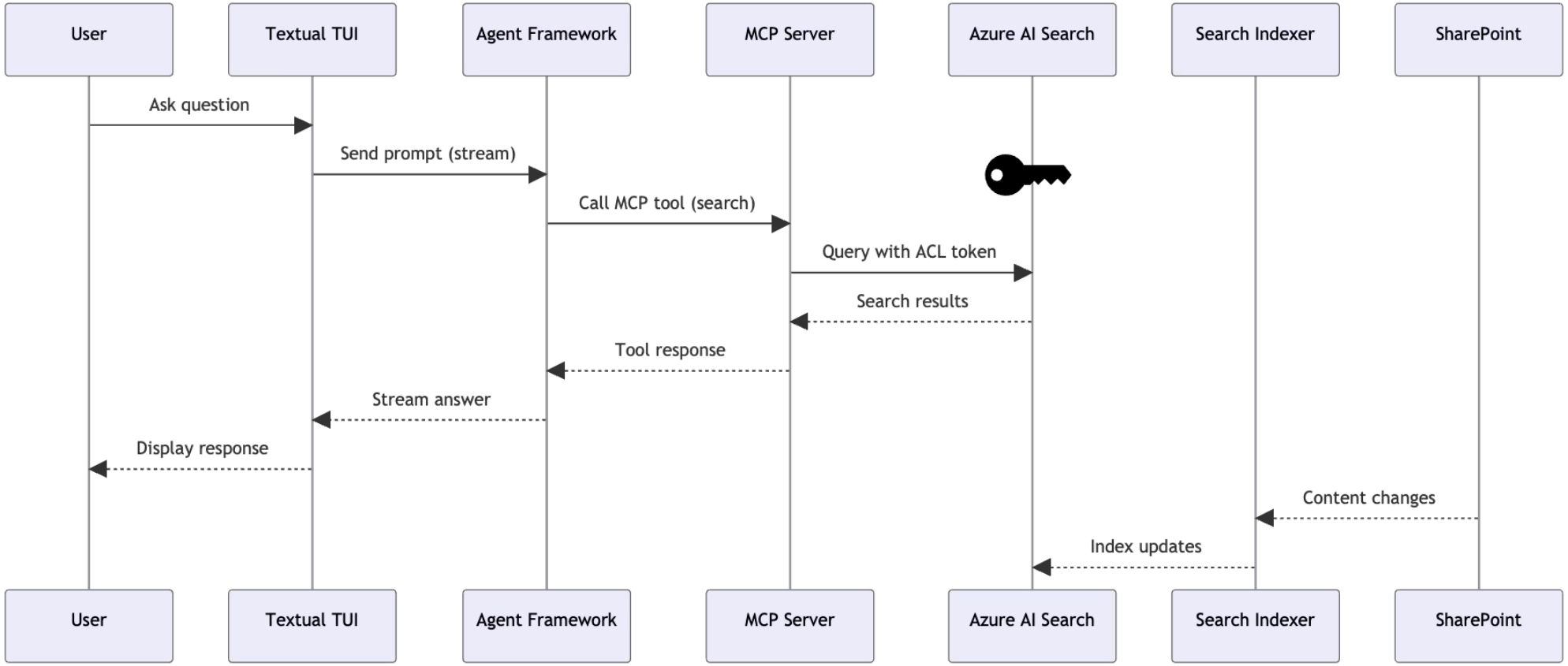
Chunk and vectorize content with Azure AI Search: pull via SharePoint data source (preview)



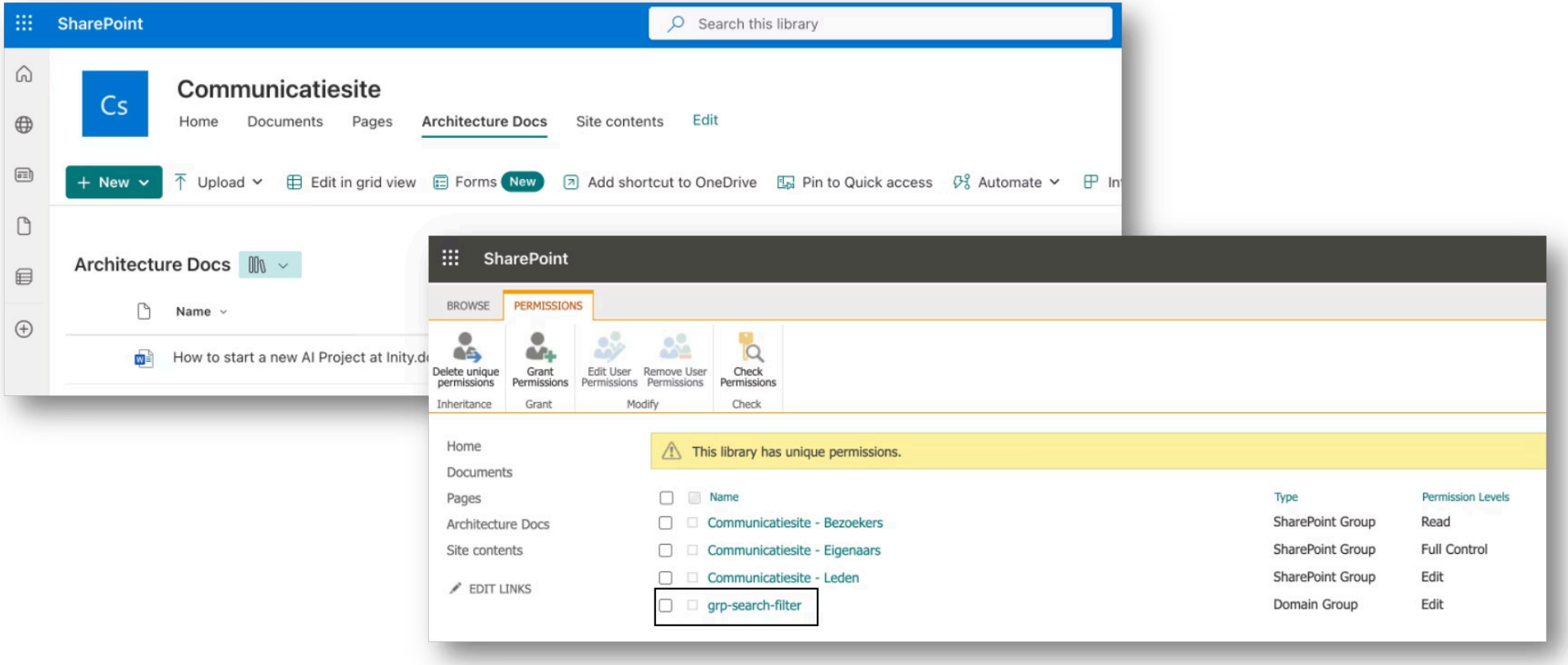
Goal is to get meaningful chunks to the agent using hybrid search with optional semantic reranker



ⓘ Agent needs user credentials → Present to MCP tool



MCP tool presents credentials to Azure AI Search for security filtering



Use domain groups in SharePoint: Entra ID object IDs need to be saved in Azure AI Search

Azure AI Search indexer can automatically populate user and group Ids (preview)

The Agent

- Agent written in Microsoft Agent Framework (.NET or Python)
- Agent is wrapped in an API protected with OAuth
- Agent uses tool on MCP server and presents Authorization header

```python
instructions = (
    "You answer questions using the MCP search tool. "
    "Always call the MCP tool to retrieve relevant documents before answering. "
    "If no documents are found, say you could not find relevant content."
)

async with (
    MCPStreamableHTTPTool(
        name="SharePoint Search MCP",
        url=MCP_SERVER_URL,
        headers=headers,
    ) as mcp_tool,
    ChatAgent(
        chat_client=chat_client,
        name="SharePointDocsAgent",
        instructions=instructions,
    ) as agent,
):
    result = await agent.run(query, tools=mcp_tool)
```

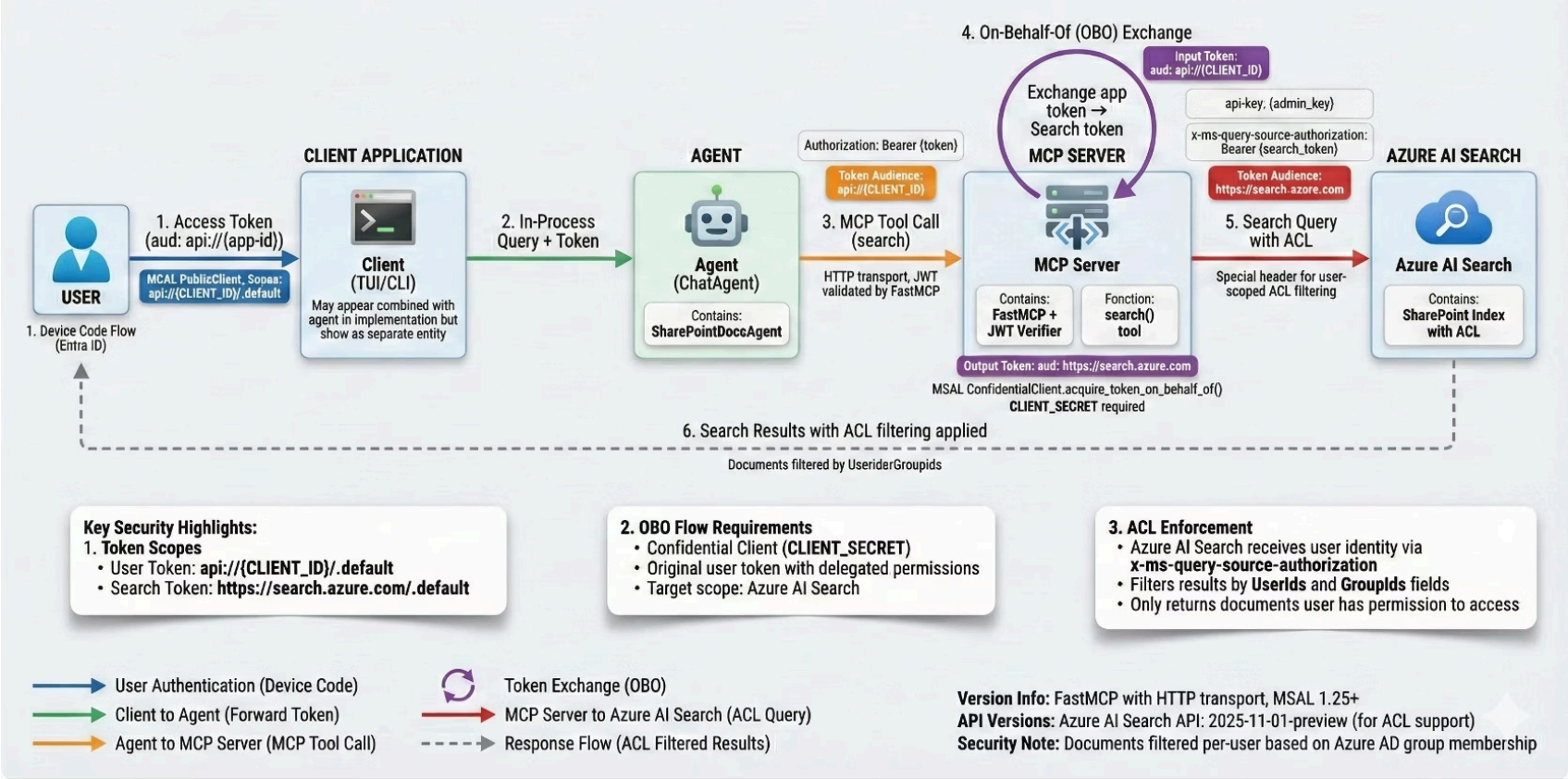
MCP Server

- Agent calls the search tool
- MCP server verifies the token
- Search tool exchanges app token for Azure AI Search token
- MCP tool calls the Azure AI Search APIs with token in a special header to apply filtering

```python
url = f"{search_endpoint}/indexes/{resource_prefix}-index/docs/search"
body = {
    "search": query,
    "select": "snippet,metadata_title,metadata_spo_item_name,metadata_spo_item_weburi",
    "top": top,
    "count": True,
}
headers = {
    "api-key": api_key,
    "x-ms-query-source-authorization": search_token,
    "Content-Type": "application/json",
}

try:
    response = post_with_retry(
        url,
        params={"api-version": api_version},
        headers=headers,
        body=body,
    )
except requests.exceptions.RequestException as exc:
    return {"authenticated": True, "error": str(exc)}
```

End-to-end flow



ⓘ Many variations possible: e.g. tool could live with the agent (no MCP)

PART 2

Agent Layer

A typical agent with Microsoft Agent Framework

```
using System;
using Azure.AI.Projects;
using Azure.Identity;
using Microsoft.Agents.AI;
using Microsoft.Extensions.AI;

var endpoint = Environment.GetEnvironmentVariable("AZURE_OPENAI_ENDPOINT")
    ?? throw new InvalidOperationException("Set AZURE_OPENAI_ENDPOINT");
var deploymentName = Environment.GetEnvironmentVariable("AZURE_OPENAI_DEPLOYMENT_NAME") ?? "gpt-4o-
mini";
AIAgent agent = new AIProjectClient(new Uri(endpoint), new DefaultAzureCredential())
    .AsAIAgent(
        model: deploymentName,
        instructions: "You are a helpful assistant.",
        tools: [AIFunctionFactory.Create(GetWeather)]);
```

```
GitHub Copilot v1.0.24
Describe a task to get started.

Tip: /diff Review the changes made in the current directory
Copilot uses AI. Check for mistakes.

● 💡 No copilot instructions found. Run /init to generate a copilot-instructions.md file for this project.

● Environment loaded: 3 MCP servers, 1 plugin, 45 skills

~/scratch/dotnetcommunity                                   GPT-5.3-Codex (high)
❯ Type @ to mention files, # for issues/PRs, / for commands, or ? for shortcuts
shift+tab switch mode                                       Remaining reqs.: 98.4%
```

What Is an Agent Harness?

An agent harness is a **ready-to-go toolkit** for building smart, long-running AI agents. Instead of manually wiring up planning, file access, memory, and context management from scratch, a harness gives you all of these **out of the box.**

> Think of it like buying a fully assembled car vs. sourcing individual parts — same engine under the hood, dramatically less work to get moving.

🧠 Planning

Built-in task decomposition

📁 File & Shell Access

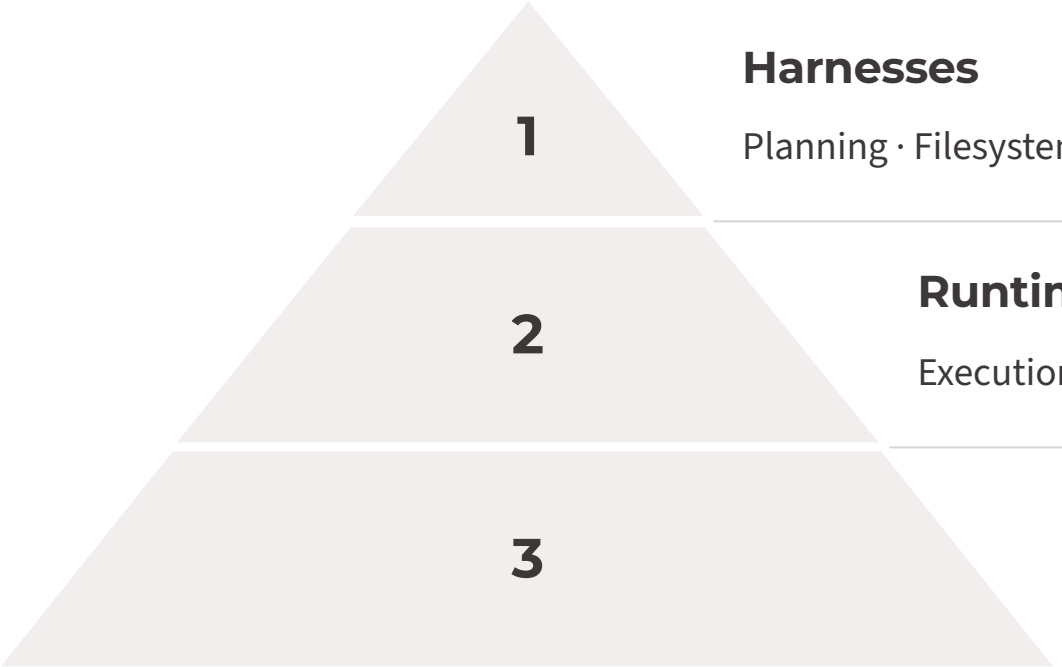
Virtual filesystem
Execute commands

🧩 Memory

Context managed automatically

Where Harnesses Fit in the Agent Stack

Three distinct layers — each with a different job. Harnesses own the top.



Harnesses

Planning · Filesystem · Subagents · Context — higher level features

Runtimes

Execution, streaming, durability

Frameworks

LangChain · MAF — Core building blocks

Frameworks give you building blocks. Runtimes handle execution and durability. **Harnesses sit on top of both** — adding everything your agent needs to handle real-world complexity without reinventing the wheel.

Harnesses in the Wild

The model is increasingly commodity. Claude, GPT, and Gemini perform similarly at baseline. What separates great agents from broken ones is the harness.

🤖 **Claude Code** — Anthropic's harness for long-running coding tasks. Built-in context compaction, tool use, and subagent support out of the box.

💻 **OpenAI Codex CLI** — OpenAI's published account of harness engineering: ~1M lines of code across ~1,500 PRs in 5 months with a small team.

🌐 **Manus** — Rebuilt their harness 5 times in 6 months. Meta acquired them for ~$2B in Dec 2025 — not for the model, but for the harness.

Deep Agents SDK

LangChain's answer to Claude Code — a batteries-included harness built on top of LangChain and LangGraph. Provider-agnostic, opinionated, and ready to ship.

📋 **Planning** — write_todos tool for structured task decomposition before acting

📁 **Virtual Filesystem** — pluggable backends: in-memory, local disk, cloud stores, sandboxes

🤝 **Subagents** — spawn focused workers for task delegation

🧠 **Context Compression** — offloads large tool results, summarizes conversation history automatically

🛡️ **Human-in-the-Loop** — approval gates for sensitive operations

📝 **Persistent Memory** — **AGENTS.md** files for cross-session knowledge

ⓘ Uses LangGraph under the hood

Why Use a Harness?

Every serious agent needs the same set of primitives. You can build them yourself — or you can start with a harness that already has them. The harness is the difference between shipping in a week and shipping in a quarter. Maybe… 😊

- → **Complex multi-step task execution**
- → **Long-running sessions with large context**
- → **Subagent delegation and parallelism**
- → **Persistent memory across conversations**
- → **Safe execution of untrusted code**

The model is commodity. The harness has more value.

Claude, GPT, and Gemini perform similarly at baseline. What separates great agents from broken ones isn't the model — it's the planning, memory, and orchestration wrapped around it.

Harnesses in the .NET world

Microsoft Agent Framework has built-in support for these patterns **https://devblogs.microsoft.com/agent-framework/agent-harness-in-agent-framework/**

Use GitHub Copilot SDK or Claude Code as an agent harness directly from Agent Framework

Or use .NET SDK for GitHub Copilot directly

Support for Agent Skills in MAF

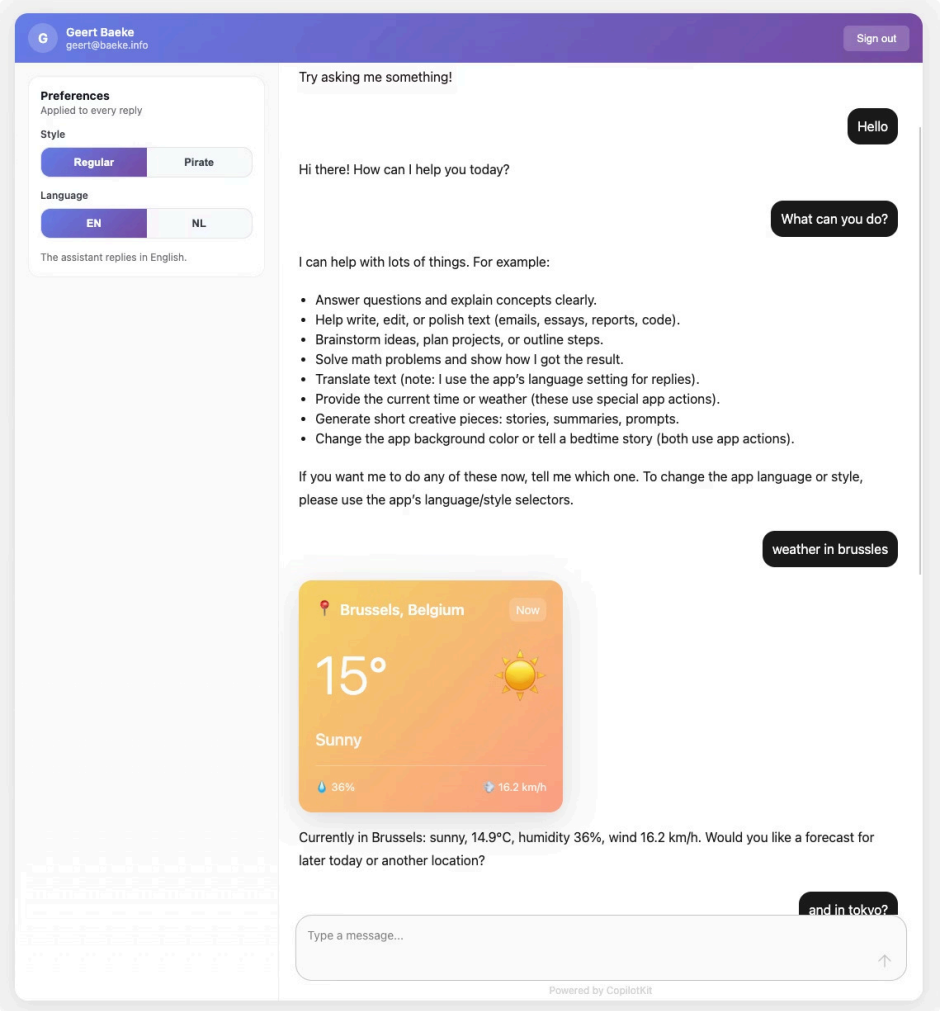
Demo

See it in action

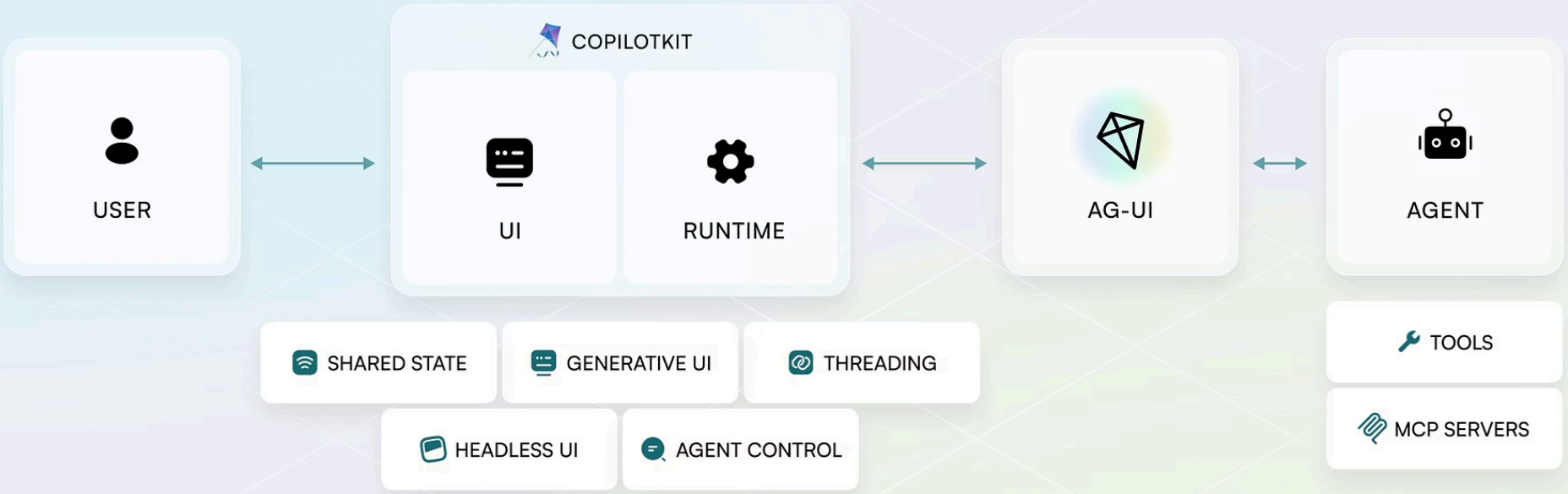
SRE Agent

PART 3

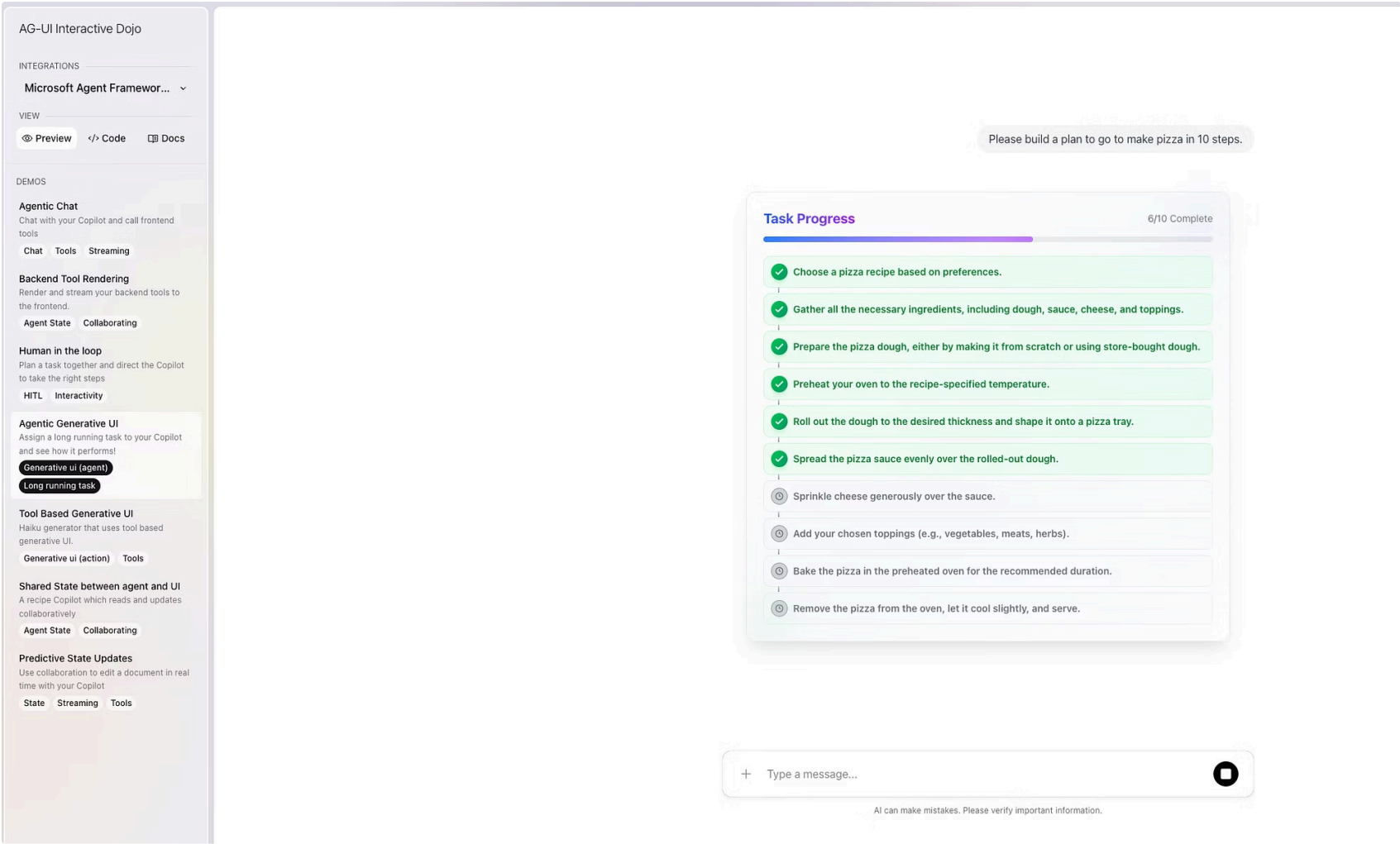
UI Layer


G
Geert Baeke
geert@baeke.info
Sign out
Preferences
Applied to every reply
Style
Regular
Pirate
Language
EN
NL
The assistant replies in English.
Try asking me something!
Hello
Hi there! How can I help you today?
What can you do?
I can help with lots of things. For example:
• Answer questions and explain concepts clearly.
• Help write, edit, or polish text (emails, essays, reports, code).
• Brainstorm ideas, plan projects, or outline steps.
• Solve math problems and show how I got the result.
• Translate text (note: I use the app's language setting for replies).
• Provide the current time or weather (these use special app actions).
• Generate short creative pieces: stories, summaries, prompts.
• Change the app background color or tell a bedtime story (both use app actions).
If you want me to do any of these now, tell me which one. To change the app language or style, please use the app's language/style selectors.
weather in brussles
Brussels, Belgium
Now
15°
Sunny
36%
16.2 km/h
Currently in Brussels: sunny, 14.9°C, humidity 36%, wind 16.2 km/h. Would you like a forecast for later today or another location?
and in tokyo?
Type a message...
Powered by CopilotKit

Agent Interaction Flow



CopilotKit



Demo Viewer by CopilotKit

How this all fits together?

Build sophisticated long-running agents with an **agent harness or build from ground up**

Provide the agent with enterprise retrieval tools with **ACL support** in Azure AI Search

Build the UI with **AG-UI and CopilotKit** using built-in support for the agent framework/harness